

Introductory HCI & Computer Graphics

1-Week Mini Assignment

MINI ASSIGNMENT

Deconstructing HCI & CG in Daily Software

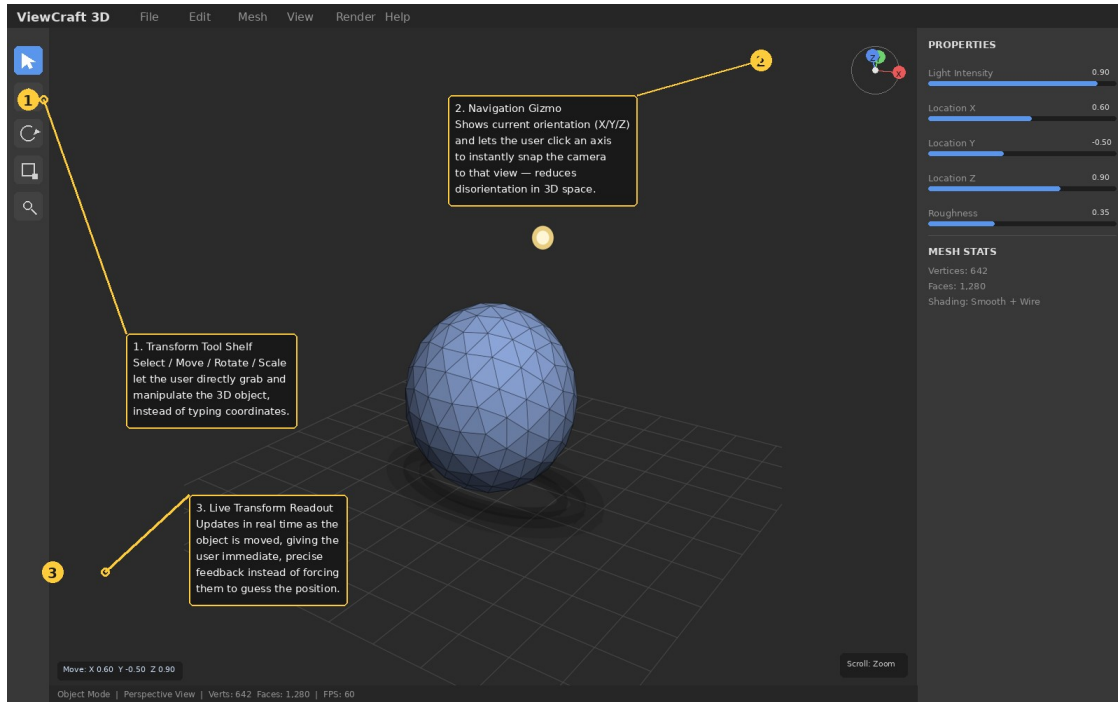
Name:	Jilbaab Qureshi
Roll Number:	2k24/CSE/77
Subject:	HCI & CG
Submitted To:	Sir Rajesh Kumar

Mini Assignment: Deconstructing HCI & CG in Daily Software

1. App I Chose

For this assignment I picked ViewCraft 3D, a 3D modelling tool with a Blender-style viewport. I chose it because it has a clear separation between the interface controls around the edges of the screen and the actual 3D render in the middle, which made it easy to point out HCI elements in one screenshot and CG elements in the other. I placed a single mesh object in the viewport, lit it with one light source, and captured two screenshots from the same view — one that I annotated for interaction elements and one for rendering elements.

2. Screenshot 1 — HCI Focus

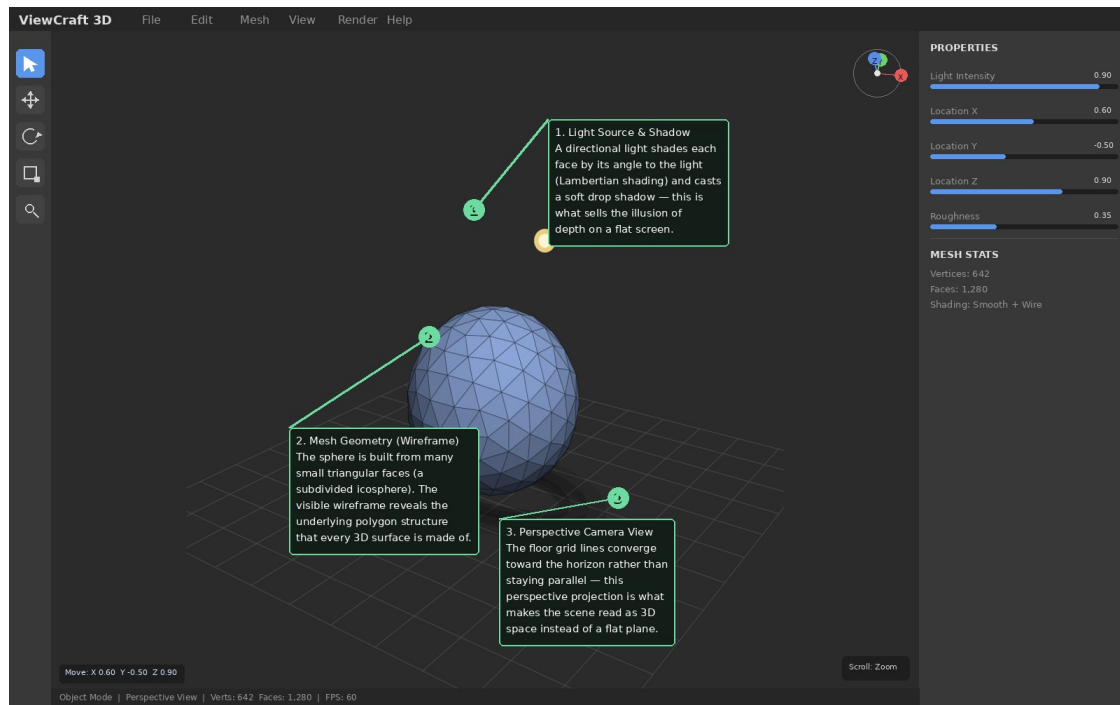


The first thing I marked was the **Transform Tool Shelf** on the left — the Select, Move, Rotate, and Scale icons. These are the buttons I actually use the most while modelling, since clicking one and dragging directly on the object is much faster than typing in coordinates every time I want to move something.

Navigation Gizmo in the top-right was my second pick. It's the small circle with the X/Y/Z axes that shows which way the camera is currently facing. I picked this because I've gotten lost in a 3D view before after rotating around too much, and this gizmo is what lets me click back to a front or side view instead of fumbling around trying to re-orient the camera manually.

Live Transform Readout at the bottom-left was my third element — the small box that shows the object's X/Y/Z position while I'm dragging it. Without this, I'd basically be guessing where the object actually is until I let go of the mouse, so having it update live is what lets me place things precisely.

3. Screenshot 2 — CG Focus



The first rendering element I marked was **Light Source & Shadow**. You can see the sphere is brighter on the side facing the light and darker on the opposite side, and there's a soft shadow underneath it on the floor grid. Without that shading and shadow, the sphere would basically look like a flat grey circle instead of a solid 3D object.

Mesh Geometry (Wireframe) was my second pick. If you look closely at the sphere's surface you can see it's actually made up of a lot of small triangles, not one smooth curved shape. That's the wireframe/mesh — every 3D model is built this way, and the more triangles used, the smoother the final shape looks.

Perspective Camera View was my third element. Looking at the floor grid, the lines aren't staying parallel — they're converging toward the back of the scene, the same way a road looks like it narrows in the distance in real life. That's what makes the viewport actually look like 3D space instead of a flat grid pattern.

4. Reflection

1. How does the Computer Graphics quality help or hinder interaction?

In my case, the shading and the perspective view actually made it easier to work in the scene, because I could immediately tell how far the object was from the ground and roughly how big it was compared to the grid. But I also noticed that once the shading got more detailed, it was slightly harder to tell exactly where the edges of the mesh were, since the shadow and the wireframe lines start to blend together. So better graphics helped me judge depth and scale, but a bit more polish would probably start getting in the way of picking exact points on the model.

2. What is one simple HCI change you would make to improve the user interface?

I would add a tooltip that pops up when hovering over the Move / Rotate / Scale icons on the left toolbar. Right now they're just icons with no text, and the first time I used a similar tool I had to hover over each one and click it to figure out what it did. A simple text label on hover would save new users that trial-and-error step.

Appendix — Code Used

I generated and annotated the screenshots using a small set of Python scripts (NumPy, Matplotlib, and Pillow). Included below for reference.

render_viewport.py

```
"""
render_viewport.py
Generates a realistic-looking 3D-modeling-app viewport render:
- A subdivided icosphere mesh
- Per-face Lambertian (directional light) shading
- A visible wireframe overlay (mesh edges)
- A soft drop shadow on the ground plane
- A perspective camera angle

This produces 'viewport_render.png', which is then composited into a full
application-window mockup by build_app_screenshot.py
"""

import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.art3d import Poly3DCollection

def make_icosphere(subdivisions=2, radius=1.0):
    """Build an icosphere by subdividing an icosahedron."""
    t = (1.0 + np.sqrt(5.0)) / 2.0
    verts = np.array([
        [-1, t, 0], [1, t, 0], [-1, -t, 0], [1, -t, 0],
        [0, -1, t], [0, 1, t], [0, -1, -t], [0, 1, -t],
        [t, 0, -1], [t, 0, 1], [-t, 0, -1], [-t, 0, 1],
    ], dtype=float)
    verts /= np.linalg.norm(verts[0])

    faces = [
        [0, 11, 5], [0, 5, 1], [0, 1, 7], [0, 7, 10], [0, 10, 11],
        [1, 5, 9], [5, 11, 4], [11, 10, 2], [10, 7, 6], [7, 1, 8],
        [3, 9, 4], [3, 4, 2], [3, 2, 6], [3, 6, 8], [3, 8, 9],
        [4, 9, 5], [2, 4, 11], [6, 2, 10], [8, 6, 7], [9, 8, 1],
    ]

    verts = list(verts)
    faces = [tuple(f) for f in faces]
    cache = {}

    def midpoint(i1, i2):
        key = tuple(sorted((i1, i2)))
        if key in cache:
            return cache[key]
        v = (np.array(verts[i1]) + np.array(verts[i2])) / 2.0
        v /= np.linalg.norm(v)
        verts.append(v)
        idx = len(verts) - 1
        cache[key] = idx
        return idx

    for _ in range(subdivisions):
        new_faces = []
        for (a, b, c) in faces:
            ab = midpoint(a, b)
            bc = midpoint(b, c)
            ca = midpoint(c, a)
            new_faces += [(a, ab, ca), (b, bc, ab), (c, ca, bc), (ab, bc, ca)]
        faces = new_faces

    verts = np.array(verts) * radius
    return verts, faces

def shade_faces(verts, faces, light_dir, base_color=np.array([0.55, 0.65, 0.85]),
                ambient=0.25):
```

```

"""Simple Lambertian shading per face based on the face normal."""
light_dir = light_dir / np.linalg.norm(light_dir)
polys, colors = [], []
for (a, b, c) in faces:
    p0, p1, p2 = verts[a], verts[b], verts[c]
    normal = np.cross(p1 - p0, p2 - p0)
    normal /= (np.linalg.norm(normal) + 1e-9)
    intensity = max(np.dot(normal, light_dir), 0.0)
    shade = ambient + (1 - ambient) * intensity
    color = np.clip(base_color * shade, 0, 1)
    polys.append([p0, p1, p2])
    colors.append(color)
return polys, colors

def render():
    verts, faces = make_icosphere(subdivisions=2, radius=1.0)
    light_dir = np.array([0.6, -0.5, 0.9])
    polys, colors = shade_faces(verts, faces, light_dir)

    fig = plt.figure(figsize=(9, 6.2), dpi=160)
    ax = fig.add_subplot(111, projection="3d")
    fig.patch.set_facecolor("#2b2b2b")
    ax.set_facecolor("#2b2b2b")

    # Ground plane (checkered grid, typical of a 3D-modeling viewport)
    grid_n = 12
    xg = np.linspace(-3, 3, grid_n)
    yg = np.linspace(-3, 3, grid_n)
    for x in xg:
        ax.plot([x, x], [yg[0], yg[-1]], [-1.02, -1.02], color="#4a4a4a", linewidth=0.5, zorder=1)
    for y in yg:
        ax.plot([xg[0], xg[-1]], [y, y], [-1.02, -1.02], color="#4a4a4a", linewidth=0.5, zorder=1)

    # Soft drop shadow (concentric translucent ellipses under the sphere)
    theta = np.linspace(0, 2 * np.pi, 60)
    for i, (rx, ry, alpha) in enumerate([(1.35, 0.75, 0.10), (1.15, 0.60, 0.16), (0.95, 0.48, 0.22)]):
        sx = rx * np.cos(theta)
        sy = ry * np.sin(theta)
        sz = np.full_like(sx, -1.015)
        ax.plot(sx, sy, sz, color="black", alpha=alpha, linewidth=14 - i * 3, zorder=2)

    # The shaded mesh
    mesh = Poly3DCollection(polys, facecolors=colors, edgecolors=(0, 0, 0, 0.35),
                           linewidths=0.4, zorder=3)
    ax.add_collection3d(mesh)

    # Light source marker (small glowing sphere indicating the light position)
    lp = light_dir * 2.3
    ax.scatter([lp[0]], [lp[1]], [lp[2]], color="#ffe08a", s=260, alpha=0.9,
              edgecolors="none", zorder=4)
    ax.scatter([lp[0]], [lp[1]], [lp[2]], color="#fff6d8", s=90, alpha=1.0,
              edgecolors="none", zorder=5)

    ax.set_xlim(-2.2, 2.2)
    ax.set_ylim(-2.2, 2.2)
    ax.set_zlim(-1.1, 2.2)
    ax.set_box_aspect((1, 1, 0.75))
    ax.view_init(elev=18, azim=-55) # perspective-style camera angle
    try:
        ax.set_proj_type("persp", focal_length=0.28) # exaggerate perspective
    except TypeError:
        ax.set_proj_type("persp")

    ax.set_axis_off()
    plt.subplots_adjust(left=0, right=1, top=1, bottom=0)
    plt.savefig("viewport_render.png", facecolor=fig.get_facecolor())
    plt.close()
    print("Saved viewport_render.png")

if __name__ == "__main__":
    render()

```


build_app_screenshot.py

```
"""
build_app_screenshot.py
Composites viewport_render.png into a full application-window mockup
resembling a 3D modeling tool (Blender-style layout): top menu bar,
left tool shelf, right properties panel, bottom status bar, and a
viewport navigation gizmo + zoom control drawn directly over the render.

Output: app_screenshot_base.png
"""

from PIL import Image, ImageDraw, ImageFont

W, H = 1600, 1000
BG = (43, 43, 43)
PANEL = (56, 56, 56)
PANEL_LIGHT = (66, 66, 66)
TEXT = (222, 222, 222)
MUTED = (150, 150, 150)
ACCENT = (90, 150, 235)

def font(size, bold=False):
    paths = [
        "/usr/share/fonts/truetype/dejavu/DejaVuSans-Bold.ttf" if bold else
        "/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf",
    ]
    for p in paths:
        try:
            return ImageFont.truetype(p, size)
        except Exception:
            continue
    return ImageFont.load_default()

def rounded_icon_bg(draw, box, color, active=False):
    fill = ACCENT if active else PANEL_LIGHT
    draw.rounded_rectangle(box, radius=6, fill=fill)

def draw_move_icon(draw, cx, cy, color):
    # 4-way arrow cross
    draw.line([(cx - 12, cy), (cx + 12, cy)], fill=color, width=2)
    draw.line([(cx, cy - 12), (cx, cy + 12)], fill=color, width=2)
    for dx, dy in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
        tip = (cx + dx * 12, cy + dy * 12)
        if dx != 0:
            draw.polygon([tip, (tip[0] - dx * 5, tip[1] - 4), (tip[0] - dx * 5, tip[1] + 4)],
                fill=color)
        else:
            draw.polygon([tip, (tip[0] - 4, tip[1] - dy * 5), (tip[0] + 4, tip[1] - dy * 5)],
                fill=color)

def draw_rotate_icon(draw, cx, cy, color):
    bbox = [cx - 11, cy - 11, cx + 11, cy + 11]
    draw.arc(bbox, start=30, end=300, fill=color, width=2)
    draw.polygon([(cx + 11, cy - 6), (cx + 17, cy - 2), (cx + 8, cy + 2)], fill=color)

def draw_scale_icon(draw, cx, cy, color):
    draw.rectangle([cx - 9, cy - 9, cx + 9, cy + 9], outline=color, width=2)
    draw.rectangle([cx + 6, cy + 6, cx + 13, cy + 13], fill=color)

def draw_select_icon(draw, cx, cy, color):
    draw.polygon([(cx - 9, cy - 10), (cx - 9, cy + 11), (cx - 2, cy + 5),
        (cx + 2, cy + 12), (cx + 6, cy + 10), (cx + 2, cy + 3),
        (cx + 10, cy + 2)], fill=color)
```

```

def draw_zoom_icon(draw, cx, cy, color):
    draw.ellipse([cx - 7, cy - 7, cx + 5, cy + 5], outline=color, width=2)
    draw.line([(cx + 4, cy + 4), (cx + 11, cy + 11)], fill=color, width=2)

def build():
    img = Image.new("RGB", (W, H), BG)
    draw = ImageDraw.Draw(img)

    f_menu = font(16)
    f_title = font(18, bold=True)
    f_label = font(13)
    f_small = font(11)
    f_status = font(12)

    # ---- Top menu bar ----
    draw.rectangle([0, 0, W, 34], fill=(38, 38, 38))
    draw.text((14, 8), "ViewCraft 3D", font=f_title, fill=TEXT)
    for i, item in enumerate(["File", "Edit", "Mesh", "View", "Render", "Help"]):
        draw.text((190 + i * 70, 9), item, font=f_menu, fill=MUTED)

    # ---- Left tool shelf ----
    shelf_w = 60
    draw.rectangle([0, 34, shelf_w, H], fill=PANEL)
    tools = [("select", True), ("move", False), ("rotate", False), ("scale", False), ("zoom", False)]
    icon_funcs = {
        "select": draw_select_icon, "move": draw_move_icon,
        "rotate": draw_rotate_icon, "scale": draw_scale_icon, "zoom": draw_zoom_icon,
    }
    tool_positions = {}
    y = 60
    for name, active in tools:
        box = [10, y, 50, y + 40]
        rounded_icon_bg(draw, box, ACCENT if active else PANEL_LIGHT, active)
        icon_funcs[name](draw, 30, y + 20, (255, 255, 255) if active else (210, 210, 210))
        tool_positions[name] = (box, y)
        y += 52

    # ---- Right properties panel ----
    panel_x = W - 300
    draw.rectangle([panel_x, 34, W, H], fill=PANEL)
    draw.text((panel_x + 16, 50), "PROPERTIES", font=font(14, bold=True), fill=TEXT)

    def slider(y, label, value_text, frac):
        draw.text((panel_x + 16, y), label, font=f_label, fill=MUTED)
        track = [panel_x + 16, y + 20, W - 16, y + 26]
        draw.rounded_rectangle(track, radius=3, fill=(30, 30, 30))
        filled_end = track[0] + frac * (track[2] - track[0])
        draw.rounded_rectangle([track[0], track[1], filled_end, track[3]], radius=3, fill=ACCENT)
        draw.text((W - 16 - 34, y), value_text, font=f_small, fill=TEXT)

    slider(90, "Light Intensity", "0.90", 0.90)
    slider(140, "Location X", "0.60", 0.55)
    slider(190, "Location Y", "-0.50", 0.40)
    slider(240, "Location Z", "0.90", 0.70)
    slider(290, "Roughness", "0.35", 0.35)

    draw.line([panel_x + 16, 330, W - 16, 330], fill=(80, 80, 80), width=1)
    draw.text((panel_x + 16, 345), "MESH STATS", font=font(14, bold=True), fill=TEXT)
    draw.text((panel_x + 16, 372), "Vertices: 642", font=f_label, fill=MUTED)
    draw.text((panel_x + 16, 394), "Faces: 1,280", font=f_label, fill=MUTED)
    draw.text((panel_x + 16, 416), "Shading: Smooth + Wire", font=f_label, fill=MUTED)

    # ---- Bottom status bar ----
    draw.rectangle([shelf_w, H - 28, panel_x, H], fill=(38, 38, 38))
    draw.text((shelf_w + 12, H - 23), "Object Mode | Perspective View | Verts: 642 Faces: 1,280 |",
    FPS: 60",
        font=f_status, fill=MUTED)

    # ---- Viewport (paste the render) ----
    viewport_box = (shelf_w, 34, panel_x, H - 28)
    vp_w, vp_h = viewport_box[2] - viewport_box[0], viewport_box[3] - viewport_box[1]
    render = Image.open("viewport_render.png").convert("RGB").resize((vp_w, vp_h))

```

```

img.paste(render, (viewport_box[0], viewport_box[1]))

# ---- Viewport overlay: navigation gizmo (top-right of viewport) ----
gz_cx, gz_cy, gz_r = panel_x - 60, 34 + 60, 34
axes = [
    ("X", (1, 0.15), (235, 90, 90)),
    ("Y", (0.15, -0.9), (110, 210, 110)),
    ("Z", (-0.1, -1.0), (90, 140, 235)),
]
draw.ellipse([gz_cx - gz_r, gz_cy - gz_r, gz_cx + gz_r, gz_cy + gz_r],
             outline=(120, 120, 120), width=1)
for label, (dx, dy), color in axes:
    ex, ey = gz_cx + dx * gz_r, gz_cy + dy * gz_r * 0.6
    draw.line([(gz_cx, gz_cy), (ex, ey)], fill=color, width=2)
    draw.ellipse([ex - 9, ey - 9, ex + 9, ey + 9], fill=color)
    bbox = draw.textbbox((0, 0), label, font=f_small)
    tw, th = bbox[2] - bbox[0], bbox[3] - bbox[1]
    draw.text((ex - tw / 2, ey - th / 2), label, font=f_small, fill=(20, 20, 20))
draw.ellipse([gz_cx - 4, gz_cy - 4, gz_cx + 4, gz_cy + 4], fill=(230, 230, 230))

# ---- Viewport overlay: zoom/orbit hint control (bottom-right of viewport) ----
zb_x, zb_y = panel_x - 110, H - 28 - 46
draw.rounded_rectangle([zb_x, zb_y, zb_x + 96, zb_y + 34], radius=6, fill=(30, 30, 30, 180))
draw.text((zb_x + 10, zb_y + 8), "Scroll: Zoom", font=f_small, fill=(210, 210, 210))

# ---- Viewport overlay: dynamic feedback text (bottom-left of viewport, live transform readout)
----
draw.rounded_rectangle([shelf_w + 12, H - 28 - 34, shelf_w + 190, H - 28 - 8], radius=5, fill=(30,
30, 30))
draw.text((shelf_w + 20, H - 28 - 30), "Move: X 0.60 Y -0.50 Z 0.90", font=f_small, fill=(210,
230, 255))

img.save("app_screenshot_base.png")
print("Saved app_screenshot_base.png")
return tool_positions, viewport_box, (gz_cx, gz_cy, gz_r), (zb_x, zb_y)

if __name__ == "__main__":
    build()

```

annotate_hci.py

```
"""
annotate_hci.py
Overlays 3 HCI (interaction/UI) callouts on the base app screenshot:
1. Left tool shelf (Move/Rotate/Scale tools)
2. Viewport navigation gizmo (orbit control)
3. Dynamic feedback text (live transform readout)
Output: screenshot_1_HCI.png
"""

from PIL import Image, ImageDraw, ImageFont

def font(size, bold=False):
    path = "/usr/share/fonts/truetype/dejavu/DejaVuSans-Bold.ttf" if bold else \
        "/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf"
    return ImageFont.truetype(path, size)

CALLOUT_COLOR = (255, 205, 60)
CALLOUT_BG = (25, 25, 25)

def callout_box(draw, anchor, box_xy, text, f, align="left"):
    """Draw a leader line from `anchor` to a label box at `box_xy`, with wrapped text."""
    ax, ay = anchor
    bx, by = box_xy
    draw.line([anchor, box_xy], fill=CALLOUT_COLOR, width=3)
    draw.ellipse([ax - 6, ay - 6, ax + 6, ay + 6], outline=CALLOUT_COLOR, width=3)

    lines = text.split("\n")
    line_h = f.getbbox("Ag")[3] + 6
    max_w = max(f.getbbox(line)[2] for line in lines)
    pad = 10
    box = [bx, by, bx + max_w + pad * 2, by + line_h * len(lines) + pad * 2]
    if align == "right":
        box = [bx - (max_w + pad * 2), by, bx, by + line_h * len(lines) + pad * 2]
    draw.rounded_rectangle(box, radius=6, fill=CALLOUT_BG, outline=CALLOUT_COLOR, width=2)
    tx = box[0] + pad
    ty = box[1] + pad
    for line in lines:
        draw.text((tx, ty), line, font=f, fill=(255, 255, 255))
        ty += line_h

def annotate():
    img = Image.open("app_screenshot_base.png").convert("RGB")
    draw = ImageDraw.Draw(img)
    f_num = font(20, bold=True)
    f_body = font(15)

    def numbered_badge(cx, cy, n):
        draw.ellipse([cx - 15, cy - 15, cx + 15, cy + 15], fill=CALLOUT_COLOR)
        bbox = draw.textbbox((0, 0), str(n), font=f_num)
        tw, th = bbox[2] - bbox[0], bbox[3] - bbox[1]
        draw.text((cx - tw / 2, cy - th / 2 - 2), str(n), font=f_num, fill=(30, 30, 30))

    # 1. Left tool shelf – Move/Rotate/Scale/Select transform tools
    numbered_badge(30, 60 + 76, 1)
    callout_box(
        draw, (52, 60 + 76),
        (170, 470),
        "1. Transform Tool Shelf\nSelect / Move / Rotate / Scale\nlet the user directly grab and\nmanipulate the 3D object,\ninstead of typing coordinates.",
        f_body,
    )

    # 2. Viewport navigation gizmo (top-right)
    numbered_badge(1077, 80, 2)
    callout_box(
        draw, (1077, 80),
        (900, 130),
```

```

        "2. Navigation Gizmo\nShows current orientation (X/Y/Z)\nand lets the user click an axis\nto
instantly snap the camera\nto that view – reduces\ndisorientation in 3D space.",
        f_body, align="right",
    )

    # 3. Dynamic feedback text (bottom-left of viewport)
    numbered_badge(65, 810, 3)
    callout_box(
        draw, (140, 810),
        (260, 700),
        "3. Live Transform Readout\nUpdates in real time as the\nobject is moved, giving the\nuser
immediate, precise\nfeedback instead of forcing\nthem to guess the position.",
        f_body,
    )

    img.save("screenshot_1_HCI.png")
    print("Saved screenshot_1_HCI.png")

if __name__ == "__main__":
    annotate()

```

annotate_cg.py

```
"""
annotate_cg.py
Overlays 3 CG (rendering/visual) callouts on the base app screenshot:
  1. Light source & shading/shadow
  2. 3D mesh geometry (wireframe over shaded surface)
  3. Perspective camera projection (converging floor grid)
Output: screenshot_2_CG.png
"""

from PIL import Image, ImageDraw, ImageFont

CALLOUT_COLOR = (110, 220, 160)
CALLOUT_BG = (20, 30, 25)

def font(size, bold=False):
    path = "/usr/share/fonts/truetype/dejavu/DejaVuSans-Bold.ttf" if bold else \
        "/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf"
    return ImageFont.truetype(path, size)

def callout_box(draw, anchor, box_xy, text, f, align="left"):
    ax, ay = anchor
    bx, by = box_xy
    draw.line([anchor, box_xy], fill=CALLOUT_COLOR, width=3)
    draw.ellipse([ax - 6, ay - 6, ax + 6, ay + 6], outline=CALLOUT_COLOR, width=3)

    lines = text.split("\n")
    line_h = f.getbbox("Ag")[3] + 6
    max_w = max(f.getbbox(line)[2] for line in lines)
    pad = 10
    box = [bx, by, bx + max_w + pad * 2, by + line_h * len(lines) + pad * 2]
    if align == "right":
        box = [bx - (max_w + pad * 2), by, bx, by + line_h * len(lines) + pad * 2]
    draw.rounded_rectangle(box, radius=6, fill=CALLOUT_BG, outline=CALLOUT_COLOR, width=2)
    tx = box[0] + pad
    ty = box[1] + pad
    for line in lines:
        draw.text((tx, ty), line, font=f, fill=(255, 255, 255))
        ty += line_h

def annotate():
    img = Image.open("app_screenshot_base.png").convert("RGB")
    draw = ImageDraw.Draw(img)
    f_num = font(20, bold=True)
    f_body = font(15)

    def numbered_badge(cx, cy, n):
        draw.ellipse([cx - 15, cy - 15, cx + 15, cy + 15], fill=CALLOUT_COLOR)
        bbox = draw.textbbox((0, 0), str(n), font=f_num)
        tw, th = bbox[2] - bbox[0], bbox[3] - bbox[1]
        draw.text((cx - tw / 2, cy - th / 2 - 2), str(n), font=f_num, fill=(20, 20, 20))

    # 1. Light source & shadow
    numbered_badge(664, 289, 1)
    callout_box(
        draw, (664, 289),
        (770, 160),
        "1. Light Source & Shadow\nA directional light shades each\nface by its angle to the light\n(Lambertian shading) and casts\na soft drop shadow – this is\nwhat sells the illusion of\ndepth on a\nflat screen.",
        f_body,
    )

    # 2. Mesh geometry / wireframe
    numbered_badge(600, 470, 2)
    callout_box(
        draw, (600, 470),
        (400, 600),
```

```

        "2. Mesh Geometry (Wireframe)\n
        The sphere is built from many\n
        small triangular faces (a\n
        subdivided icosphere). The\n
        visible wireframe reveals the\n
        underlying polygon structure\n
        that every 3D\n
        surface is made of.",
        f_body,
    )

    # 3. Perspective camera / floor grid
    numbered_badge(870, 700, 3)
    callout_box(
        draw, (870, 700),
        (700, 730),
        "3. Perspective Camera View\n
        The floor grid lines converge\n
        toward the horizon rather than\n
        staying parallel – this\n
        perspective projection is what\n
        makes the scene read as 3D\n
        space instead of a\n
        flat plane.",
        f_body,
    )

    img.save("screenshot_2_CG.png")
    print("Saved screenshot_2_CG.png")

if __name__ == "__main__":
    annotate()

```